Detecting Illicit Bitcoin Transactions with Label-Consistent Graph Stacking

BT4012 In-class Kaggle Competition 2026 · e1304439 · Code: github.com/emmyyyli/BT4012-Kaggle-Competition (submission notebook `e1304439.ipynb`)

1. Introduction

Criminals use Bitcoin to launder money and pay for illegal goods. The competition asks for a model that ranks Bitcoin transactions by their probability of being illicit. The data is a Bitcoin transaction graph: each node is a transaction described by 165 anonymised features, and each edge is a payment flow, where an output of one transaction is spent by another. Models train on time steps 1–35 and are scored by ROC-AUC on the later steps 36–49. This mirrors anti-money-laundering (AML) work: investigators need a risk ranking that catches illicit flows early without flooding them with false alarms.

The problem has three properties, and each one shaped the approach:

- **Scarce, imbalanced labels.** Only 31,235 of 141,772 training transactions are labeled, and 11.7% of those are illicit. Supervised learning therefore uses labeled rows only, with balanced class weights.
- **Temporal drift.** The test period lies in the future; a classifier separates training rows from test rows with AUC ≈ 1.0 from the features alone. Validation is therefore always forward in time, training is weighted toward recent steps, and `time_step` is never a model input.
- **Graph structure with missing context.** Neighbouring transactions are informative, but in the test period most neighbours have no features. Every graph feature is therefore built from information that exists in the same form at training and prediction time.

The final model is a two-stage LightGBM stack. A first model scores each transaction from its own features; a second model adds summaries of its neighbours in the payment graph. These summaries use only *labeled* neighbours, the only transactions with features in both the training and test periods, so they are computed the same way at training and prediction time. No transaction's own label is used to build its features. The model raised the public ROC-AUC from 0.94246 (first submission) to **0.97097**.

2. Methodology

2.1 Data preprocessing and feature engineering

Audit. The supplied features have no missing values and no duplicate IDs. Though anonymised, they form two blocks: features correlate more within `feat_1`–`feat_93` and within `feat_94`–`feat_165` (mean |r| 0.12 and 0.07) than across them (0.04), and the second block drifts far more. We label them "local" and "aggregated" (the usual layout of transaction-graph data, where the second block summarises the neighbourhood); the data supports two blocks, not this meaning, and the final model uses all 165 features. Rows labeled `none` (110,537) were treated as **unlabeled, not licit**. Using them as pseudo-labels lowered time-forward AUC by about 0.03, because unlabeled transactions are not a random sample of labeled ones. `time_step` is used only to split data. As a feature it lowered long-horizon AUC (0.878 vs 0.884), and trees cannot extrapolate a counter to unseen steps.

The decisive finding: the test period has blank neighbourhoods.

- **How it was found:** matching edge-list IDs to the feature tables left 46,668 graph nodes with no features, all in steps 36–49. Test rows are scored against labels, so they are that period's *labeled* transactions; the blank nodes are its unlabeled ones, which in training appear as `none` rows *with* features.
- **What it means:** 65.6% of test-node edges lead to a featureless node, against 0% in training, so the same neighbourhood looks complete in training and mostly blank at test.
- **Why all neighbours cannot be used:** a model that learns from complete neighbourhoods meets different inputs at test time. Our GraphSAGE blend, which read all neighbours, dropped the public score from 0.94246 to 0.91059.
- **How it is managed:** graph features use only labeled neighbours, which have features in both periods, and two-hop walks pass *through* blank nodes to reach labeled ones (below).

Feature drift. Medians of 19.4% of the aggregated features shift by more than one interquartile range from training to test, against 9.7% within the training period. Local features barely move (1.1%). An aggregated-only model scored 0.880 on Kaggle. Removing the five most-drifted features was neutral (0.94365 vs 0.94417), and correlated-cluster ablation, SHAP-stability subsets and adversarial feature dropping gave no gain, so all 165 were kept.

Label-consistent graph features.

- **Motivation:** labeled transactions are strongly homophilous. A labeled neighbour of an illicit transaction is illicit 57.4% of the time, against a 11.7% base rate.
- **Design rule:** context is built only from *labeled* neighbours, the only transactions with features in both periods, and no target label is ever used. With this rule, coverage is similar in training and test (Figure 1). Features, each added as a separately tested step:

1. **Neighbour scores:** mean, max, incoming-mean and outgoing-mean of stage-1 predictions over labeled neighbours.
2. **Neighbourhood structure:** in/out degree, plus the number and share of labeled neighbours.
3. **Neighbour feature statistics:** mean, max, std, incoming-mean and outgoing-mean over labeled neighbours of the 25 most influential supplied features, ranked by test-row SHAP of the base model (no labels used).

4. **Two-hop context:** walk-weighted means of stage-1 scores and the same 25 features over labeled nodes two steps away (first 5 features, later all 25). Walks may pass *through* featureless nodes, because the edge list is complete in both periods.
5. **Payment-chain shape** (label-free, full edge list): longest payment path into and out of the transaction, and the length of the unbroken *peeling chain* around it (one funder, at most two spenders), a classic laundering pattern. Each time step is exactly one connected component, so component-level counts were excluded: they would encode the time step.

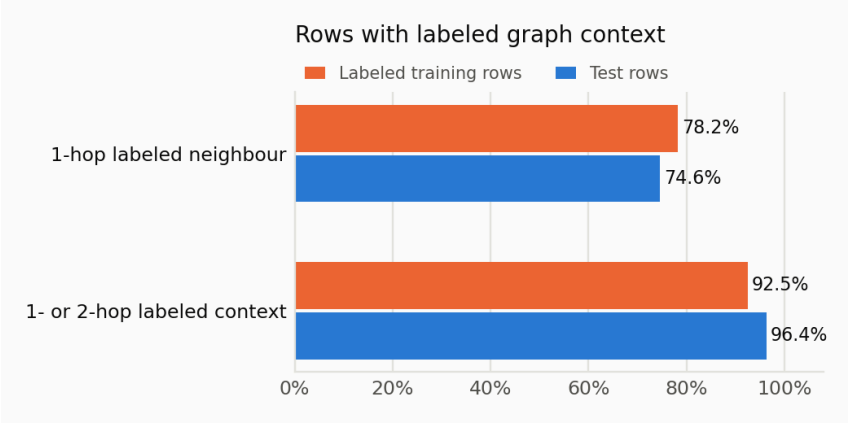


Figure 1. Share of rows with labeled graph context. The one-hop rule keeps training and test comparable; two-hop walks through featureless nodes reach 21.8% of test rows that have no labeled neighbour (14.3% in training).

2.2 Model development and validation

Base learner. LightGBM, tuned with Optuna (40 trials): 9 leaves, 500 trees, learning rate 0.025, at least 49 rows per leaf, 64% of features and 98% of rows per tree, L2 penalty 2.0. Every search, including XGBoost and CatBoost, preferred shallow trees (4–9 leaves or depth 3–4): regularisation is the main defence against drift. Class imbalance is handled with `class_weight="balanced"`; ROC-AUC needs no threshold, so no resampling.

Stacking without leakage.

1. **Stage 1:** score every labeled training row *out-of-fold*, with folds as blocks of consecutive time steps. Score test rows with the mean of the five fold models, so that training and test scores come from the same kind of model.
2. **Context:** build the graph features of §2.1 from those scores.
3. **Stage 2:** refit LightGBM on the supplied features plus the context.
4. **Weighting toward the test period:** every fit weights rows by 2^((t − 35)/14) (step 35 counts 1, step 21 counts 0.5) times a *covariate-shift importance weight*: a classifier separates training from test rows (no labels), and each training row gets √(p/(1 − p)), clipped to [0.2, 5]. With all features the periods separate perfectly (AUC 1.000) through time-trend features, which would only repeat the recency weights, so the 30 most separating features are removed for this classifier only (AUC 0.94; weights correlate 0.37 with the time step). Predictions are averaged over 10 seeds.

Validation. The original 4-fold time-series CV averaged 0.977, while Kaggle gave 0.942. Single-holdout scores also moved by up to 0.02 between random seeds, which was larger than most of the effects being tested. We therefore built a harness (`src/evaluation.py`):

- five forward windows (train ≤ t, validate a later block), each with five seeds;
- seed-averaged ROC-AUC and PR-AUC, per-step AUC, and a paired bootstrap against a reference;
- adoption rule: the mean must improve and no window may lose more than 0.01.

It had one limitation. The long windows (≈0.90 AUC) match the test's difficulty but ranked models in the *opposite* order to Kaggle. The short windows (≈0.996) ranked them correctly but are too close to the ceiling to resolve gains of 0.01. Local validation therefore served as a correctness check, and the public leaderboard (30% of the test set) acted as the final arbiter. To limit overfitting to it, every submission tested one stated hypothesis against a fixed reference, and a switch required ≥ +0.003 (about two paired standard errors).

2.3 Model exploration and final strategy

Table 1. Models explored. "Local" is the mean ROC-AUC over the five validation windows, relative to the reference used at the time.

Approach	Local evidence	Public AUC	Decision
LightGBM + `time_step` + `in_degree` (first submission)	—	0.94246	baseline
Random Forest	below LightGBM on every window	—	rejected
Tuned shallow LightGBM, 165 features	+0.010 vs deep LightGBM	0.94417	base learner
XGBoost / CatBoost / cross-family ensembles	no gain over LightGBM	—	rejected
Aggregated features only	best on long windows	0.88023	rejected (drift)

Approach	Local evidence	Public AUC	Decision
GraphSAGE blend (all neighbours)	—	0.91059	rejected (blank neighbours)
GraphSAGE with test-like masking	−0.047	—	rejected
Self-supervised graph embeddings	−0.04 (long holdout)	—	rejected
Pseudo-labeling `none` rows	−0.03	—	rejected
Recency weights (half-life 14)	≈+0.001 short windows	0.95066	adopted
Labeled-neighbour stacking	failed local rule	0.95389	adopted
+ structure & neighbour features / + recency		0.95871 / 0.96342	adopted
+ two-hop context		0.96668	adopted
+ 25-feature neighbour statistics		0.96814	adopted
3-hop score propagation	best long-window variant	0.96108	rejected
Extra round of collective classification	+0.005–0.011 long windows	0.96698	rejected (within noise)
Blend with a non-graph model		0.95328	rejected
Final model with 30 seeds / half-life 21	test rank $\rho \geq 0.998$	0.96785 / 0.96670	stability check
Rank blend of four graph models		0.96686	superseded as pick 2
Direction-split two-hop / hub-weighted means / centrality	flat on short windows	0.96787 / 0.96766 / 0.96759	rejected (within noise)
Forward-only stage-1 scores for training rows	flat on short windows	0.96503	rejected
+ peel-chain shape	flat on short windows	0.96858	pick 2
+ covariate-shift importance weights	flat on short windows	0.9702	adopted
+ two-hop means over all 25 features (**final**)	flat on short windows	**0.97097**	final (pick 1)

Two patterns decided the final strategy:

- **Added complexity without new information never helped.** This covers the extra model families, blends with non-graph models, deeper propagation and an extra round of collective classification.
- **New information, computed the same way in training and test, did.** This covers neighbours, their features and two-hop reach. Shifting training toward the test period helped independently: recency weighting twice, test-similarity weighting once more (+0.0016, within noise). Once labeled one- and two-hop context was in place, further descriptions of graph *shape* (direction, centrality, hubs, chain) moved the score by ≤ 0.0006; extending two-hop *reach* to all 25 features still added +0.0008.
- **Consistent *definitions* matter; matching noise levels does not.** Training-row neighbour scores are more accurate than test-row ones (out-of-fold AUC 0.997 vs 0.985 forward-only). Training stage 2 on noisier forward-only scores *lowered* the public AUC by 0.003: accurate scores teach the true strength of homophily.

Final picks. Kaggle scores the better of two selected submissions on the private 70%:

- **Pick 1:** the final model (0.97097), which `e1304439.ipynb` reproduces exactly.
- **Pick 2:** the model without shift weights or the wider two-hop means (0.96858). It hedges the riskiest component: shift weighting cuts the effective training size to 43% (77% with recency alone), and the last two gains are each within noise. The chain model's configuration is the best characterised: six close variants of it scored 0.9666–0.9686.

3. Results and discussion

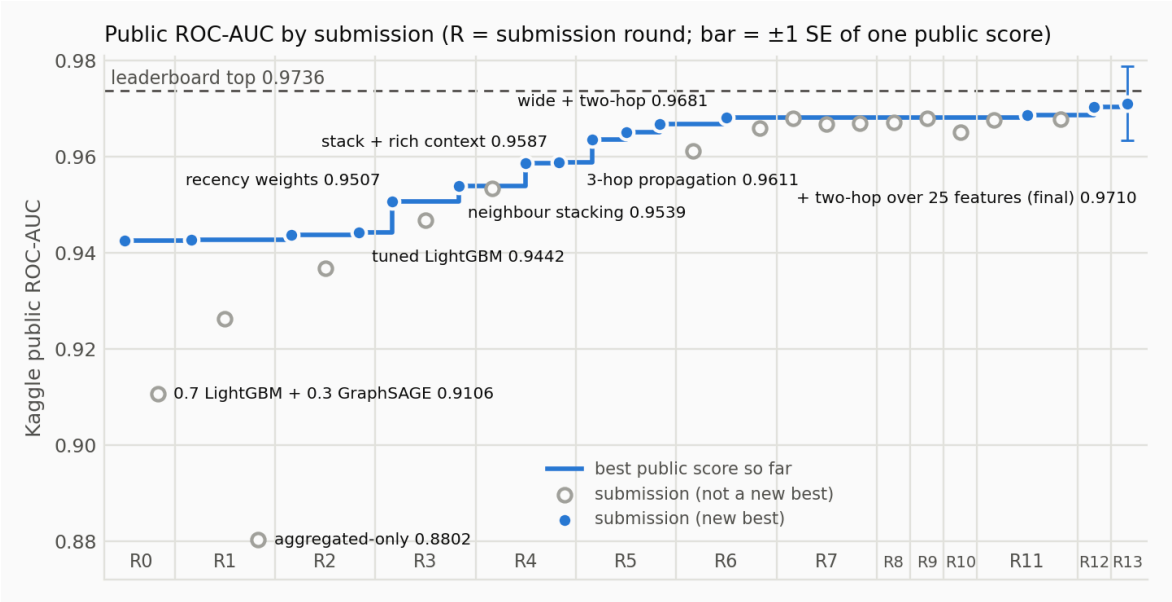


Figure 2. Public ROC-AUC of every submission. The best-so-far line rises only when new, label-consistent information is added; the ±1 SE bar (0.0077, about 270 illicit public rows) shows that the final gap to the leader is within noise.

Table 2. Final model vs the non-graph base learner on the two validation windows whose ranking matched Kaggle (3 seeds). P = precision. Validation windows are about 18% illicit, so operating points adapt to prevalence.

Window (illicit / rows)	Model	ROC-AUC	PR-AUC	P = recall at k = #illicit	P at 95% recall	Best F1
train ≤25 → 26–35 (1,307 / 7,035)	non-graph	0.9954	0.9820	0.941	0.917	0.942
	final	**0.9967**	**0.9881**	**0.946**	**0.943**	**0.949**
train ≤28 → 29–35 (1,102 / 6,028)	non-graph	0.9960	0.9809	0.946	0.940	0.947
	final	**0.9984**	**0.9941**	**0.967**	**0.983**	**0.970**

What the numbers mean.

- **On Kaggle,** graph context and test-period weighting added +0.027 public AUC over the base learner (0.94417 → 0.97097); the largest steps were neighbour stacking (+0.0097) and two-hop context (+0.0033).
- **In triage terms** (Table 2), at 95% recall false alarms among flagged transactions fall from 8.3% to 5.7% and from 6.0% to 1.7% (pick 2: 4.4% and 2.0%).
- **The model is stable.** Tripling the seeds or changing the half-life by 50% barely changes the test ranking ($\rho \geq 0.998$); the shift weights move it most (ρ 0.985), hence pick 2.
- **Two-hop context helped more on test than locally** because it reaches 21.8% of test rows with no labeled neighbour, against 14.3% in training (Figure 1).

Limitations.

- No validation window inside steps 1–35 reproduces the train→test drift or the test period's sparse labels. Graph variants *lost* on long local windows yet won on Kaggle, so close calls relied on the public leaderboard.
- One public AUC (about 4,600 rows) has a standard error of about 0.008, and with about 30 submissions the best public score is slightly optimistic; testing one stated idea per submission against a fixed reference limited this selection bias.
- Anonymised features prevent a domain explanation, and hidden test labels prevent analysing any regime change within steps 36–49.

4. Conclusion and reflection

The main lesson is that **how information is made available matters more than model complexity.**

- **What plateaued or failed:** a shallow LightGBM on the supplied features plateaued at 0.944. Neural graph models and embeddings failed because the neighbourhoods they read in training do not exist at test time.
- **What worked:** restricting graph features to labeled neighbours, the information that will genuinely exist at prediction time, turned graph structure from a liability into the largest source of gain. Recency weighting, out-of-fold stacking and disciplined, single-hypothesis validation made the gains reliable.
- **What I would improve:** validation windows that mask labels inside the validation period to imitate the test regime, so close calls need not rely on the public leaderboard; and checking the shift-weight gain across drop-list sizes and seeds.
- **Wider lesson:** in AI-driven fraud detection, a feature is only as good as its availability in production; "same information at training and serving time" is what keeps a deployed AML model honest.

References. Ke et al. (2017), *LightGBM*, NeurIPS. Akiba et al. (2019), *Optuna*, KDD. Lu & Getoor (2003), *Link-based Classification*, ICML. Hanley & McNeil (1982), *The meaning and use of the area under a ROC curve*, Radiology. Shimodaira (2000), *Improving predictive inference under covariate shift by weighting the log-likelihood function*, J. Stat. Planning and Inference.